



Universiteit Utrecht

[Faculty of Science
Information and Computing Sciences]

Introduction to Approximation Algorithms

Advanced Algorithms 2025/2026

Johan M. M. van Rooij

Til Miltzow

Solution quality Computation time	Optimal	Bound on quality	Good solution, no quality guarantee
Polynomial	Polynomial-time solution algorithms	Approximation algorithms	Construction heuristics
Super polynomial and/or no guarantee	Exact algorithms: • Tree search • Dynamic programming • Integer linear programming •	Hybrid algorithms • Column generation without complete branch-and-price	Meta heuristics: • Local search • Genetic algorithms

Solution quality	Optimal	Bound on quality	Good solution, no quality guarantee
Computation time			
Polynomial	Polynomial-time solution algorithms	Advanced Algorithms	Construction heuristics
	Advanced Algorithms		
Super polynomial and/or no guarantee	• Integer linear programming •	Hybrid algorithms • Column generation without complete branch-and-price	Meta heuristics: • Local search • Genetic algorithms

What to do if a problem is NP-complete?

- We have already seen many options to deal with NP-complete problems
 - Earlier in this course: FTP and exact exponential-time algorithms
 - Later in this course: Treewidth
 - In other courses: local search, ILP, constraint programming, ...
- Approximation algorithms are one of these options
 - An approximation algorithm is a heuristic with a performance guarantee
 - We consider polynomial-time approximation algorithms
 - Non-optimal solutions, but with some performance guarantee compared to the optimal solution
- Also useful as a starting point for other approaches:
 - Local search, branch and bound



What is a polynomial-time approximation algorithm?

An algorithm that...

1. 'Solves' (applies to) an optimisation problem
 - A minimisation problem or a maximisation problem
 - E.g., independent set, vertex cover, knapsack, max-SAT
 - Not: SAT, 3-SAT, etc.
2. Runs in polynomial time
3. It has a bound on the quality of the solution
 - This is called the approximation ratio
 - E.g.: value of algorithm result / optimal solution value $\leq c$
 - We call such an algorithm a **c-approximation**



Approximation ratio

- For an instance I of an optimisation problem, let
 - $OPT(I)$ be the value of the optimal solution to I
 - $ALG(I)$ be the value computed by the approximation algorithm
- An algorithm for a **minimisation problem** has an **approximation ratio c** , if for all instances I :

$$\frac{ALG(I)}{OPT(I)} \leq c$$

- An algorithm for a **maximisation problem** has an **approximation ratio c** , if for all instances I :

$$\frac{OPT(I)}{ALG(I)} \leq c$$



Class of approximation algorithms

- Class of problems with approximation algorithms whose approximation ratio is a constant: **APX**
 - Subclass of NP
 - We only consider optimisation problems in NP
- Other notions of approximability also exist
 - We see some of them the next lecture
 - And, one at the end of this lecture
- Notions of **APX**-completeness also exist
 - As NP-completeness
 - We will see this at the end of the lectures on approximation algorithms



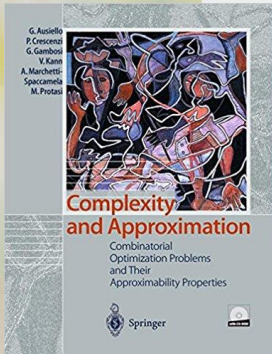
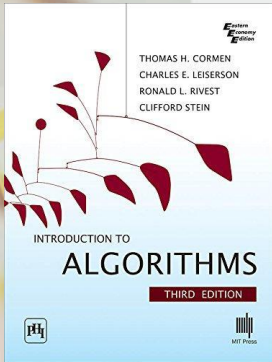
This lecture

- Approximation algorithms for a series of problems:
 - Travelling Salesman Problem
 - Minimum (Weight) Vertex Cover
 - Max Satisfiability



Further reading and additional practice

- For the design and analysis of approximation algorithms
 - Introduction to Algorithms, Cormen et al., 3th Edition, Chapter 35
 - Introduction to Algorithms, Cormen et al., 4th Edition, Chapter 35
- For the landscape of approximation algorithms
 - Complexity and Approximation. Ausiello et al., Chapter 3



Approximation algorithms – Advanced Algorithms

MAXIMUM SATISFIABILITY



Universiteit Utrecht

[Faculty of Science
Information and Computing Sciences]

Max satisfiability (decision version)

Maximum Satisfiability (Max SAT)

- Instance: A set of clauses C containing literals of variables from a set X and an integer k
- Question: Does there exist a truth assignment to the variables in X such that at least k clauses in C are satisfied?

k-Satisfiability (k-SAT):

- Instance: set of clauses C (logical formula in CNF format) where each clause contains at most k literals
- Question: does there exist a satisfying assignment to the variables in C , i.e., a truth assignment such that each clause has at least one literal set to true

Defined similarly: Maximum k-Satisfiability (Max k-SAT)



Approximating Max Satisfiability

Algorithm:

1. Take any truth assignment a to the variables X
2. Let $\neg a$ be the assignment a with all variables negated
3. Return from $\{a, \neg a\}$ the assignment that satisfies most clauses

■ This is a 2-approximation

■ $ALG \geq |C|/2, OPT \leq |C|$

■ Why?

■ Can you also give approximation algorithms for max k -Satisfiability?



Approximation algorithms – Advanced Algorithms

TRAVELLING SALESMAN PROBLEM

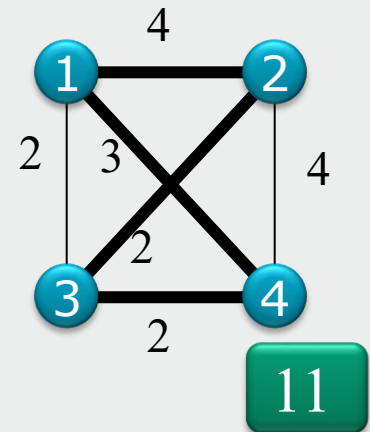
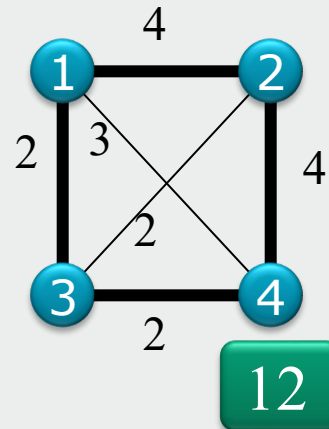
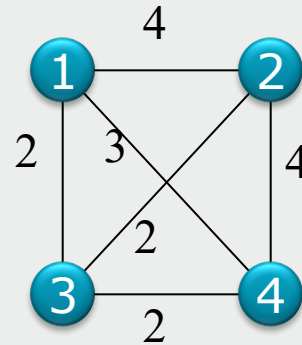


Universiteit Utrecht

[Faculty of Science
Information and Computing Sciences]

The symmetric travelling salesman problem with triangle inequality

- **Instance:** n vertices (cities), distance between every pair of vertices
- **Question:** Find shortest (simple) cycle that visits every city?
- **Restriction on distances:**
 - Symmetric: for all x, y :
 $w(x, y) = w(y, x)$
 - Triangle inequality: for all x, y, z :
 $w(x, y) + w(y, z) \geq w(x, z)$



A simple algorithm

- Consider the symmetric Travelling Salesman Problem with triangle inequality

Algorithm:

1. Find a minimum spanning tree
 2. Output vertices of tree in preorder
 - Preorder: visit a node before its children
 - Take a tour following the preorder
- This algorithm has approximation ratio 2:
 - $\text{OPT} \geq \text{MST}$
 - $2 \text{ MST} \geq \text{ALG}$
 - $\text{ALG} / \text{OPT} \leq 2\text{MST} / \text{MST} = 2$



Can we do better?

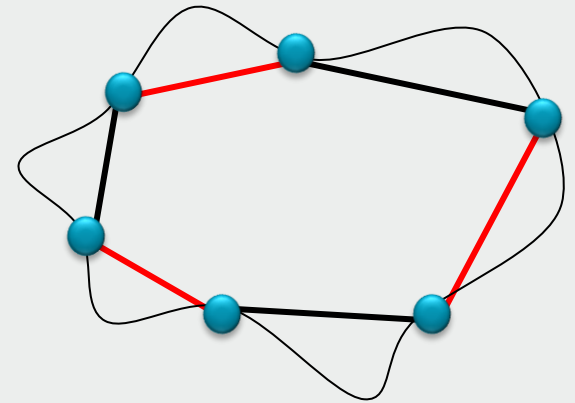
Yes: Christofides' algorithm

Christofides' Algorithm:

1. Construct a Minimum Spanning Tree T
2. Set $W = \{v \mid v \text{ has odd degree in tree } T\}$
3. Compute a minimum weight matching M in the graph $G[W]$
4. Look at the graph $T+M$
 - Note that $T+M$ is Eulerian!
5. Compute an Euler tour C' in $T+M$
6. Add shortcuts to C' to get a TSP-tour

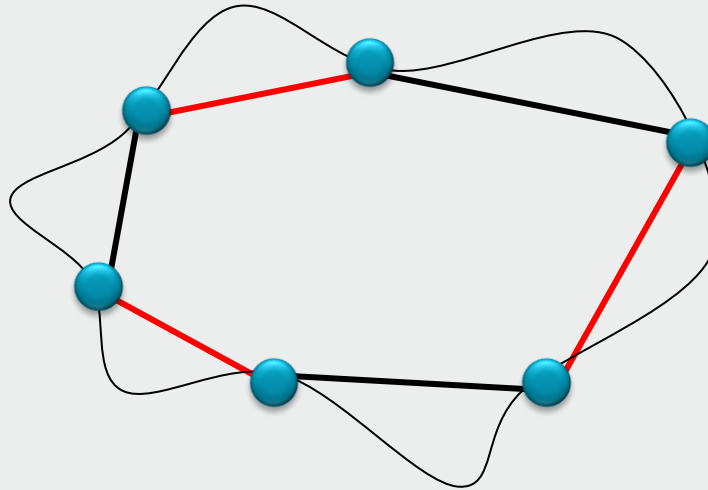
■ This is a $3/2$ -approximation

■ Proof on the blackboard



Ratio 1.5

- Total length edges in T : at most OPT
- Total length edges in matching M : at most $OPT/2$
- $T+M$ has length at most $3/2 OPT$
- Use triangle inequality



Proving approximation ratios

- We just saw two algorithms for TSP with triangle inequality
- We proved approximation ratios in the following way:
 1. Use a quantity x that relates to both the optimal solution and the result of the algorithm
 - A minimum spanning tree in both cases
 2. Prove that $\text{OPT} \geq c_1 x$
 3. Prove that $\text{ALG} \leq c_2 x$
 4. Combine both inequalities for a bound on ALG / OPT
- We will consider more examples along these lines
 - But first: the non-existence of an c -approximation on TSP **without** triangle inequality though using the gap technique



Approximation algorithms – Advanced Algorithms

THE GAP TECHNIQUE



Universiteit Utrecht

[Faculty of Science
Information and Computing Sciences]

The gap technique

■ The gap technique:

- An **NP**-hardness proof that creates a (wide) gap between YES-instance and NO-instances

■ Applied to TSP without triangle inequality:

- There is no polynomial time algorithm A for TSP without triangle inequality that approximates within a ratio c , for any constant $c > 1$
- Reduction:
 1. Take instance $G=(V,E)$ of Hamiltonian Circuit
 2. Build instance of TSP:
 - A city for each $v \in V$
 - If $(v,w) \in E$, then $d(v,w) = 1$, otherwise $d(v,w) = nc+1$
 3. A finds a tour with distance at most nc , if and only if, G has a Hamiltonian circuit



Approximation algorithms – Advanced Algorithms

MINIMUM (WEIGHT) VERTEX COVER



Universiteit Utrecht

[Faculty of Science
Information and Computing Sciences]

Approximation for minimum vertex cover

(you could have seen this one before in ADS)

Algorithm:

1. Let $E' = E$, $C = \emptyset$
2. While $E' \neq \emptyset$
 1. Let $\{u,v\}$ be any edge from E'
 2. $C := C \cup \{u,v\}$
 3. Remove every edge incident to u or v from E'
3. Return C

- Approximation algorithm?
 - Runs in polynomial time
 - Returns a vertex cover
 - How good is this vertex cover?



2-approximation for vertex cover

Theorem:

- The algorithm on the previous slide is a 2-approximation

Proof:

- Let A be the set of edges which endpoints we picked
 - $\text{OPT} \geq |A|$, because every edge in A must be covered
 - $\text{ALG} = 2|A| \leq 2\text{OPT}$, hence $\text{ALG}/\text{OPT} \leq 2$



Minimum weight vertex cover

- Minimum weight vertex cover:
 - Vertex cover where each vertex has a weight
 - We look for the minimum weight vertex cover
- 2-approximation for vertex cover no longer works
 - We may select very heavy vertices using that algorithm
- Next, we will give a 2-approximation for minimum weights vertex cover using LP rounding



An ILP formulation of the problem

- Consider the following ILP:

$$\text{minimise: } \sum_{v \in V} w_v x_v$$

$$\begin{aligned} \text{subject to: } & x_u + x_v \geq 1 && \text{for each } \{u, v\} \in E \\ & x_v \in \{0, 1\} && \text{for each } v \in V \end{aligned}$$

- In the LP relaxation of this problem, the last constraint is replaced by:

- $0 \leq x_v \leq 1$

- Linear programming can be solved in polynomial time
 - Not by the simplex algorithm!!
 - Ellipsoid method / interior point methods



2-Approximation algorithm for minimum weight vertex cover

Algorithm:

1. Compute the optimal solution to the LP relaxation
2. Output all v with $x_v \geq \frac{1}{2}$

- This algorithm returns a vertex cover
 - For every edge, sum of incident vertices at least 1
 - Hence at least one of the vertex variables at least $\frac{1}{2}$
- Approximation algorithm?
 - Runs in polynomial time and produces a vertex cover
 - How good is this vertex cover?
 - 2-Approximation proof on blackboard



Proof of 2-approximation algorithm for minimum weight vertex cover

- Let z^* be the solution to the LP
- Because any vertex cover is a solution to the LP we have:
 $z^* \leq OPT$
- Also, we can bound ALG in terms of z^* :

$$z^* = \sum_{v \in V} w_v x_v \geq \sum_{v \in V : x_v \geq \frac{1}{2}} w_v x_v \geq \sum_{v \in V : x_v \geq \frac{1}{2}} w_v \frac{1}{2} \geq \frac{1}{2} ALG$$

- Hence, $ALG \leq 2z^*$, and:

$$\frac{ALG}{OPT} \leq \frac{2z^*}{z^*} = 2$$

- QED



Approximation algorithms – Advanced Algorithms

CONCLUSION



Universiteit Utrecht

[Faculty of Science
Information and Computing Sciences]

Conclusion

- We have seen several different approximation algorithms for different problems:
 - 2-approximations for Max Satisfiability (and Max k-SAT)
 - 1.5- and 2-approximations for TSP
 - 2-approximations for vertex cover and weighted vertex cover
- c -approximations, for a constant c , are called constant factor approximation algorithms
- We have seen that NP-hardness reductions can sometimes be used to prove the non-existence of c -approximations (the gap technique)
- There are more different types of approximations
 - These we will see after the break





Universiteit Utrecht

[Faculty of Science
Information and Computing Sciences]

The Landscape of Approximation Algorithms

Advanced Algorithms 2025/2026

Johan M. M. van Rooij

Til Miltzow

What is a polynomial-time approximation algorithm?

An algorithm that...

1. Solves an optimisation problem
2. Runs in polynomial time
3. It has a bound on the quality of the solution

■ Approximation ratio c :

- $\text{ALG}/\text{OPT} \leq c$ for minimisation problems
- $\text{OPT}/\text{ALG} \leq c$ for maximisation problems
- Ratio always bigger or equal to 1



Different forms of approximation algorithms (outline of two lectures)

Qualities of polynomial-time approximation algorithms:

1. **Absolute constant difference**
 - $|\text{OPT} - \text{ALG}| \leq c$
2. **FPTAS**: Fully polynomial-time approximation scheme
 - Approximation ratio $1+\epsilon$ for any $\epsilon > 0$, while the algorithm runs in time polynomial in n and $1/\epsilon$
3. **PTAS**: Polynomial-time approximation scheme
 - Approximation ratio $1+\epsilon$ for any $\epsilon > 0$, while the algorithm runs in polynomial time for any fixed ϵ
4. **APX**: Constant-factor approximation
 - Approximation ratio: $\text{ALG}/\text{OPT} \leq c$ for minimisation problems
 - Approximation ratio: $\text{OPT}/\text{ALG} \leq c$ for maximisation problems
5. **f(n)-APX**: Approximation by a factor of $f(n)$
 - $f(n)$ depends only on the size of the input



Qualities of poly-time approximation algorithms:

1. Absolute constant difference
2. **FPTAS**: Fully polynomial-time approximation scheme
3. **PTAS**: Polynomial-time approximation scheme
4. **APX**: Constant-factor approximation
5. **f(n)-APX**: Approximation by a factor of $f(n)$

Approximation algorithms – Advanced Algorithms

ABSOLUTE CONSTANT DIFFERENCE



Absolute constant difference

- Algorithms that run in worst-case polynomial time, with:
 - $|\text{OPT} - \text{ALG}| \leq c$
- Example: Planar graph colouring
 - Algorithm that tests 1 and 2-colourability and if no solution is found outputs 4
 - Algorithm has an error of at most 1, because of the four-colour theorem

Four colour theorem:

- Any planar graph G can be coloured with at most four colours



Absolute constant difference for edge colouring

Vizings Theorem:

- Any graph G can be edge coloured with at most $\Delta(G)+1$ colours, where $\Delta(G)$ is the maximum degree of G
 - Note that any edge colouring of a graph G requires at least $\Delta(G)$ colours

Corollary:

- Edge colouring is approximable with an absolute constant error of one
 - Algorithm: Return $\Delta(G)+1$
- Still, these examples of approximation with additive constant difference are very rare!



Absolute constant difference: very rare

- Suppose we could approximate maximum independent set within an additive integer constant $c > 0$
 - Then, we can solve an NP-complete problem in polynomial time (proof on the blackboard)
- Similar construction exist for many more problems



Absolute constant difference: very rare

- Suppose we could approximate maximum independent set within an additive integer constant $c > 0$
 1. Take an instance of maximum independent set: a graph G , where we look for an independent set of size at least k
 2. Make $c + 1$ disjoint copies of G
 3. Run the approximation algorithm on the new graph
 4. If an independent set of size at least k exists in G , then the algorithm returns a solution of size $\geq (c+1)k - c$
 5. This means that it must be of size $\geq k$ on at least one disjoint copy
 6. If the independent set does not exist, then the algorithm returns a solution of size $\leq (c+1)(k-1)$
 7. This way, we can solve an NP-complete problem using this polynomial-time algorithm. That should not be possible
- Similar construction exist for many more problems



Qualities of poly-time approximation algorithms:

1. Absolute constant difference
2. **FPTAS**: Fully polynomial-time approximation scheme
3. **PTAS**: Polynomial-time approximation scheme
4. **APX**: Constant-factor approximation
5. **f(n)-APX**: Approximation by a factor of $f(n)$

Approximation algorithms – Advanced Algorithms

FULLY POLYNOMIAL-TIME APPROXIMATION SCHEME



Knapsack

- We will introduce the FPTAS (Fully Polynomial-Time Approximation Scheme) using an example first

Knapsack:

- Given: n items each with a positive integer weight $w(i)$ and integer value $v(i)$ ($1 \leq i \leq n$), and an integer W
- Question: select items with total weight at most W such that the total value is as large as possible



Dynamic programming for knapsack

- Let P be the maximum value of any item
- We can solve the problem in $O(n^2P)$ time with dynamic programming:
 - Tabulate $M(i, Q)$: minimum total weight of a subset of items $1, \dots, i$ with total value Q for Q at most nP
 - $M(0, 0) = 0$
 - $M(0, Q) = \infty$, if $Q > 0$
 - $M(i+1, Q) = \min\{ M(i, Q), M(i, Q - v(i+1)) + w(i+1) \}$
- This algorithm is clearly correct and runs in the given time
 - This is a pseudo-polynomial-time algorithm
 - Pseudo polynomial: polynomial in n and the weights P (or W)
 - Super-polynomial running time if P is (exponentially) large compared to n



Scaling for knapsack

- We will turn the pseudo-polynomial time algorithm into an approximation algorithm

Algorithm for knapsack:

1. Throw away all items that have weight larger than W
 - They are never used
2. Build a new instance:
 1. Let c be some positive constant (used to set the approx. ratio)
 2. Do not change weights, but set new values $v'(i) = \lfloor v(i) / c \rfloor$
3. Solve scaled instance with DP optimally
4. Output this solution:
 - It is an approximate solution to the original instance



The question is....

- As we scale values, not weights, the output is always feasible
- The question is: how do we set c , such that...
 - Approximation ratio good enough?
 - Polynomial time?
- Approximation quality (blackboard):
 - $ALG \geq OPT - nc$
- Running time:
 - $O(n^2P/c)$
- Next: how to choose c so that we have an FPTAS?



Approximation ratio

- Consider optimal solution Y for original problem
 - Let Y have value OPT
- Solution value scaled instance:
 - At most n items, for each $v(i)/c - \lfloor v(i)/c \rfloor < 1$
 - So, DP finds a solution of value at least $OPT/c - n$ for scaled problem instance
- Value of approximate solution for original instance is at least $c \cdot (OPT/c - n) = OPT - nc$



A different kind of approximation algorithm

- For any constant $\epsilon > 0$, we use $c = \epsilon P / (2n)$
 - Largest value of an item is at most $P/c = P / (\epsilon P / (2n)) = 2n/\epsilon$
 - Running time is $O(n^2 * 2n/\epsilon) = O(n^3/\epsilon)$
 - Approximation ratio (on blackboard): $1+\epsilon$
- Arbitrary good approximation
 - For fixed epsilon, polynomial run time
 - Run time grows exponentially in number of bits precision
- This is a **Fully Polynomial Time Approximation Scheme**
 - That is, an algorithm, that gets two inputs: the “usual” input X , and a real value $\epsilon > 0$
 - And, for each fixed $\epsilon > 0$, the algorithm is a $(1+\epsilon)$ -approximation algorithm
 - **And**, the algorithm uses polynomial time in both n and $1/\epsilon$



$(1+\varepsilon)$ -approximation

■ $OPT \geq P$

■ Each single-item set is a solution, as we removed items with weight more than W

■ Algorithm gives solution of value at least:

$$OPT - nc = OPT - n(\varepsilon P / (2n)) = OPT - \varepsilon P/2$$

$$\begin{aligned} \text{■ } OPT / (OPT - \varepsilon P/2) &\leq OPT / (OPT - \varepsilon/2 OPT) \\ &= 1 / (1 - \varepsilon/2) \leq 1+\varepsilon \end{aligned}$$

■ Where the last inequality is true if $\varepsilon \leq 1$

■ QED



Approximation algorithms – Advanced Algorithms

KNAPSACK AND THE GAP TECHNIQUE



Universiteit Utrecht

[Faculty of Science
Information and Computing Sciences]

The gap technique

■ The gap technique:

- An **NP**-hardness proof that creates a (wide) gap between YES-instance and NO-instances

■ Applied to TSP without triangle inequality:

- There is no polynomial time algorithm A for TSP without triangle inequality that approximates within a ratio c , for any constant $c > 1$
- Reduction:
 1. Take instance $G=(V,E)$ of Hamiltonian Circuit
 2. Build instance of TSP:
 - A city for each $v \in V$
 - If $(v,w) \in E$, then $d(v,w) = 1$, otherwise $d(v,w) = nc+1$
 3. A finds a tour with distance at most nc , if and only if, G has a Hamiltonian circuit



Knapsack

- We have seen an FPTAS (Fully Polynomial-Time Approximation Scheme) for knapsack last time

Knapsack:

- Given: n items each with a positive integer weight $w(i)$ and integer value $v(i)$ ($1 \leq i \leq n$), and an integer W
- Question: select items with total weight at most W such that the total value is as large as possible



Knapsack and the gap technique

- We know there exists an FPTAS for knapsack
 - If we can rule out an absolute constant difference approximation, then we know how difficult knapsack is from the viewpoint of approximation
 - Apply the gap technique: a gap creating NP-completeness proof
 - Here we assume integer valued weights and values
- Suppose an absolute constant difference approximation algorithm for knapsack exists with absolute integer error $c \geq 1$:
 1. Take an instance of knapsack
 2. Multiply the value of all items by $c+1$
 3. This does not change the set of feasible solutions
 4. However, the only solution with a difference in value of at most c from the optimal solution, is the optimal solution itself
 5. An absolute constant difference approximation will thus solve this knapsack instance optimally, contradicting $\mathbf{P} \neq \mathbf{NP}$



Qualities of poly-time approximation algorithms:

1. Absolute constant difference
2. **FPTAS**: Fully polynomial-time approximation scheme
3. **PTAS**: Polynomial-time approximation scheme
4. **APX**: Constant-factor approximation
5. **f(n)-APX**: Approximation by a factor of $f(n)$

Approximation algorithms – Advanced Algorithms

POLYNOMIAL-TIME APPROXIMATION SCHEME



PTAS and FPTAS

- A **Polynomial Time Approximation Scheme** is an algorithm, that gets two inputs: the “usual” input X , and a positive real value $\varepsilon > 0$
- For each fixed $\varepsilon > 0$, the algorithm:
 - Is an $(1+\varepsilon)$ -approximation algorithm
 - Uses polynomial time
- A **Fully Polynomial Time Approximation Scheme** is an algorithm, that gets two inputs: the “usual” input X , and a positive real value $\varepsilon > 0$
- For each $\varepsilon > 0$, the algorithm:
 - Is an $(1+\varepsilon)$ -approximation algorithm
 - The algorithm uses polynomial time in the size of X and in $1/\varepsilon$



Planar graphs and k-outerplanar graphs

- Given a planar embedding of a graph G , we define the **level** of a vertex as follows:
 - All nodes that lie on the exterior face have level 1
 - For any i , if we remove from the embedding all nodes of level smaller than i , then all nodes that lie on the border of the exterior face have level i
- A **k-outerplanar** graph is a graph that allows a planar embedding consisting of at most k levels

Lemma (that we will not prove in this lecture):

- For any k , maximum independent set restricted to k -outerplanar graphs can be solved optimally in $O(8^k n)$ time
 - k -outerplanar graphs have treewidth at most $3k-1$



PTAS for maximum independent set on planar graphs

Algorithm (input: graph G , approximation ratio $r > 1$)

1. Let $k = \lceil 1/(r-1) \rceil$
 2. Compute a planar embedding of G
 3. Compute the level numbers for all vertices in G
 4. For $i = 0$ up to and including k :
 1. Let V_i be V with all nodes that modulo $k+1$ have level i removed
 2. Let G_i be the graph induced by V_i (note: it is k -outerplanar)
 3. Compute a maximum independent set in G_i using the lemma
 5. Return the largest independent set found over all k tries
- Consider the maximum independent set I in G
- At least one $V \setminus V_i$ has $|(V \setminus V_i) \cap I| \leq |I|/(k+1)$
 - Thus, the resulting sets is of size at least: $|I| \cdot k/(k+1)$
 - Approximation ratio r



PTAS for maximum independent set on planar graphs

Algorithm (input: graph G , approximation ratio $r > 1$)

1. Let $k = \lceil 1/(r-1) \rceil$
2. Compute a planar embedding of G
3. Compute the level numbers for all vertices in G
4. For $i = 0$ up to and including k :
 1. Let V_i be V with all nodes that modulo $k+1$ have level i removed
 2. Let G_i be the graph induced by V_i (note: it is k -outerplanar)
 3. Compute a maximum independent set in G_i using the lemma
5. Return the largest independent set found over all k tries

■ Running time: $O(8^k kn)$

- Since $k = \lceil 1/(r-1) \rceil$, this is polynomial for a fixed approximation ratio r



Approximation algorithms – Advanced Algorithms

THE POLYNOMIAL-BOUND TECHNIQUE



Universiteit Utrecht

[Faculty of Science
Information and Computing Sciences]

PTAS that are not an FPTAS

- It is not always possible to construct an FPTAS (even if we have a PTAS)
- **Example:** maximum independent set has no FPTAS while we have a PTAS on a planar graphs
- 1. Suppose that it has, for each fixed $\epsilon > 0$, some algorithm that:
 - Is an $(1+\epsilon)$ -approximation algorithm
 - Uses polynomial time in the size of X and in $1/\epsilon$
- 2. Let $\epsilon = 1/n$ and run the algorithm
 - The algorithm uses polynomial time
 - The algorithm solves the problem optimally as:

$$\frac{OPT}{ALG} \leq 1 + \frac{1}{n} = \frac{n+1}{n}$$
$$ALG \geq OPT \frac{n}{n+1} = OPT \frac{n+1-1}{n+1} = OPT - \frac{OPT}{n+1} > OPT - 1$$

- 3. Hence, no such algorithm exists unless $P=NP$



PTAS that are not FPTAS are often found in a geometric setting

- Proof that no FPTAS exists also works if we restrict the class of graphs
 - The only property of max independent set that we have used is that the optimum is bounded by n
- The same proof works for any problem with a polynomial bound $p(n)$ on the size/quality of an optimal solution
 - Use $\epsilon = 1/p(n)$
 - That's why we call this the 'polynomial-bound technique'
- There exists a PTAS for maximum independent set restricted to:
 - Planar graphs
 - Unit-disk graphs (we will not consider these in the lectures)



The polynomial bound technique

■ Generic form of the polynomial bound technique

1. Suppose that the problem has an FPTAS, then for each fixed $\varepsilon > 0$, there is some algorithm that:
 - Is a $(1+\varepsilon)$ -approximation algorithm
 - Uses polynomial time in the size of X and in $1/\varepsilon$
2. Let $\varepsilon = 1/p(n)$ and run the algorithm
 - The algorithm uses polynomial time
 - The algorithm solves the problem optimally as:

$$\frac{OPT}{ALG} \leq \frac{p(n) + 1}{p(n)} \Rightarrow ALG \geq OPT \frac{p(n)}{p(n) + 1} = OPT - \frac{OPT}{p(n) + 1} > OPT - 1$$

3. Hence, no such algorithm exists unless $P=NP$



Qualities of poly-time approximation algorithms:

1. Absolute constant difference
2. **FPTAS**: Fully polynomial-time approximation scheme
3. **PTAS**: Polynomial-time approximation scheme
4. **APX**: Constant-factor approximation
5. **$f(n)$ -APX**: Approximation by a factor of $f(n)$

Approximation algorithms – Advanced Algorithms

APX



APX: constant factor approximation

- **APX:** class of problems for which approximation algorithms exists with a constant approximation ratio
 - This is different from the PTAS / FPTAS setting where we can get arbitrarily good approximations
 - Problems with an (F)PTAS are in APX
- We have seen enough examples of this last week also
 - 2-approximation for Max Satisfiability (and Max k-SAT)
 - 1.5- and 2-approximations for TSP
 - 2-approximations for vertex cover and weighted vertex cover
- We will give negative examples after first considering the last class of approximation algorithms



No PTAS for maximum independent set

Next lecture on approximation complexity we will discuss **APX-completeness**

- The hardest problems from APX w.r.t. (not) having a PTAS

Theorem:

Maximum independent set is **APX-hard** (last lecture)

Corollary:

- There is no approximation algorithm for maximum independent set that, for each fixed $\varepsilon > 0$:
 - Is an $(1+\varepsilon)$ -approximation algorithm
 - Uses polynomial time



Qualities of poly-time approximation algorithms:

1. Absolute constant difference
2. **FPTAS**: Fully polynomial-time approximation scheme
3. **PTAS**: Polynomial-time approximation scheme
4. **APX**: Constant-factor approximation
5. **$f(n)$ -APX**: Approximation by a factor of $f(n)$

Approximation algorithms – Advanced Algorithms

$F(N)$ -APX



Approximation by a factor of $f(n)$

- Approximation ratio of $f(n)$
 - Approximation ratio depends on the size of the input
 - (and can be very bad)

Set Cover

- Given: finite set U (of size n) and a family F of subsets of U
- Question: Does there exist a subset C of F of size at most k such that $\bigcup_{S \in C} S = U$?

Greedy algorithm:

- While $U \neq \emptyset$:
 - Select $S \in F$ that maximises $|S \cap U|$
 - Add S to C and let $U := U \setminus S$



An $(\ln(n)+1)$ -approximation algorithm for set cover

Greedy algorithm:

- While $U \neq \emptyset$:
 - Select $S \in F$ that maximises $|S \cap U|$
 - Add S to C and let $U := U \setminus S$

Theorem:

- The greedy set cover algorithm is an $(\ln(n)+1)$ -approximation

Proof:

1. The algorithm runs in polynomial time and returns a set cover
2. Let S_i be the i^{th} set from F picked by the algorithm
3. Let C_i be the then covered elements: $C_i = (S_1 \cup S_2 \cup \dots \cup S_{i-1})$
4. We assign cost $c(x)$ to elements $x \in U$:
 - Let x be firstly covered by set S_i while running the algorithm
 - Define: $c(x) = 1 / |S_i \setminus C_i|$
 - Now we have:

$$ALG = \sum_{x \in U} c(x)$$

5. ...



Bounding the cost of any set S

Greedy algorithm:

- While $U \neq \emptyset$:
 - Select $S \in F$ that maximises $|S \cap U|$
 - Add S to C and let $U := U \setminus S$

- We will prove that for any set S :
$$\sum_{x \in S} c(x) \leq \sum_{i=1}^{|S|} \frac{1}{i}$$
- Let $S = \{s_1, s_2, \dots, s_k\}$ with the elements ordered so that s_k is the first element that is covered by the algorithm and s_1 the last element
 - When we cover s_i in step j of the algorithm we have: $|S_j \setminus C_j| \geq i$
As we select the set with most uncovered elements
 - Hence: $c(s_i) = \frac{1}{|S_j \setminus C_j|} \leq \frac{1}{i}$



Proof of the $(\ln(n)+1)$ -approximation algorithm for set cover

- The n^{th} Harmonic number $H(n)$ is defined as: $H(n) = \sum_{i=1}^n \frac{1}{i}$
 - From mathematics (bounding using integrals):

$$H(n) = \sum_{i=1}^n \frac{1}{i} \leq \ln(n) + 1$$

- We just proved that for any $S \in \mathcal{F}$: $\sum_{x \in S} c(x) \leq \sum_{i=1}^{|S|} \frac{1}{i} = H(|S|)$

- From this we derive our approximation ratio:
 - Where C^* is the optimal set cover.

$$ALG = \sum_{x \in U} c(x) \leq \sum_{S \in C^*} \sum_{x \in S} c(x) \leq |C^*| \cdot H\left(\max_{S \in \mathcal{F}} |S|\right) \leq OPT \cdot (\ln(n) + 1)$$



Approximation algorithms – Advanced Algorithms

HARDNESS OF MAXIMUM INDEPENDENT SET



Universiteit Utrecht

[Faculty of Science
Information and Computing Sciences]

Recall:

No PTAS for maximum independent set

Corollary of APX-hardness (last lecture):

- There is no approximation algorithm for maximum independent set that, for each fixed $\varepsilon > 0$:
 - Is an $(1+\varepsilon)$ -approximation algorithm
 - Uses polynomial time
- Next step is to prove that we cannot find a c -approximation for maximum independent set
 - We will use a self reduction



Self reduction for maximum independent set

Lemma:

- If independent set can be approximated with approximation ratio c , then it can be approximated with ratio $\sqrt{c}$

Proof:

1. Given a graph G , we construct $G \times G$:
 1. It has nodes (u,v) corresponding to tuples of vertices from G
 2. It has edge $(u,v)-(u',v')$ if and only if:
 - $u = u'$, and (v,v') an edge in G
 - (u,u') an edge in G
2. Claim G has an independent set of size k , if and only if, $G \times G$ has an independent set of size k^2
3. Run the c -approximation on $G \times G$ with result J :
 1. The result in $G \times G$ has $\text{OPT}^2 \leq c |J|$
 2. Thus in G : $\text{OPT} \leq \sqrt{c} \sqrt{|J|}$



Proof of claim

Claim:

- G has an independent set of size $\geq k$, if and only if,
 $G \times G$ has an independent set of size $\geq k^2$

Proof:

1. Given an independent set I in G of size k , we construct one in $G \times G$ of size k^2 by taking all (v,u) for $v \in I$ and $u \in I$
2. Let J be an independent set in $G \times G$ of size $\geq k^2$
 1. Consider all u with $(u,v) \in J$
 2. If these vertices form an independent set of size $\geq k$ in G then we are done
 3. Otherwise, there are less than k different such u
 4. So for at least one fixed u , there must be more than k different $(u,v) \in J$
 5. These vertices v form an independent set in G of size $\geq k$



Maximum independent set is not in APX

- If independent set can be approximated with approximation ratio c , then it can be approximated with ratio $\sqrt{c}$

Corollary:

- If independent set can be approximated with approximation ratio c , then it can be approximated with ratio $1 + \epsilon$ (it has a PTAS)
 - For fixed ϵ , we can repeat the square root trick often enough (but a fixed number of times) until the ratio is good enough

Corollary:

- Maximum independent set is not in **APX**
 - It has no PTAS assuming **P**≠**NP**, hence also no c -approximation



Concluding remarks on independent set

- Maximum independent set is in $f(n)$ -**APX**, where:

$$f(n) = \frac{n}{\log^2 n}$$

- Things can be even worse:
 - TSP without triangle inequality cannot be approximated by any polynomial-time approximation algorithm with an approximation ratio $p(n)$ for any polynomial $p(n)$ unless **P=NP**



Approximation algorithms – Advanced Algorithms

CONCLUSION



Universiteit Utrecht

[Faculty of Science
Information and Computing Sciences]

Conclusion on approximation

Qualities of polynomial-time approximation algorithms:

1. Absolute constant difference
 2. **FPTAS**: Fully polynomial-time approximation scheme
 3. **PTAS**: Polynomial-time approximation scheme
 4. **APX**: Constant-factor approximation
 5. **f(n)-APX**: Approximation by a factor of $f(n)$
- Proving that a quality of approximation algorithm 'most likely' does not exist:
- Gap technique
 - Polynomial-bound technique
 - **APX**-completeness / **APX**-hardness (next lecture)

